

Ingeniería de Software II

Estilos de arquitectura

Documentación

- ▶ Una descripción de la arquitectura es un conjunto de artefactos que documentan la arquitectura de tal manera que los stakeholders puedan:
 - ▶ Entender la arquitectura.
 - ▶ Validar la arquitectura.
 - ▶ Criticar la arquitectura.
 - ▶ Proponer mejoras.

Documentación

- ▶ Preguntas que un arquitecto debe responder:
 - ¿Cuáles son los principales elementos funcionales de su arquitectura?
 - ¿Cómo estos elementos interactuarán con otros?
 - ¿Qué información puede ser manejada, persistida y presentada?
 - ¿Qué elementos de software y hardware pueden ser requeridos para soportar estos elementos funcionales y de información?
 - ¿Qué características operacionales y capacidades serán previstas?
 - ¿Qué ambientes de desarrollo, test, soporte y training serán necesarios?

Detalle requerido

- ▶ Es muy importante lograr el nivel de detalle adecuado en la descripción de la arquitectura.
 - Visión general
 - Detalle particular
 - Permite validar y desarrollar
- ▶ Evitar crear una descripción de la arquitectura que incluya TODO.
 - ¿Por qué?

Muchas maneras

- ▶ Hay diversos “estándares” para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Views and Viewpoints

Muchas maneras

- ▶ Hay diversos “estándares” para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

- Idea central: la arquitectura de un **sistema es demasiado compleja para describirla desde un único ángulo**. Por eso se documenta mediante vistas (views) y puntos de vista (viewpoints).
- View: una representación de la arquitectura, hecha con un objetivo y para un grupo de interés (ej. vista lógica, vista de procesos, vista de despliegue).
- Viewpoint: la plantilla o “receta” que define cómo construir una vista: qué **notación** usar, qué **elementos** incluir, **para quién** está pensada.

La vista lógica es un view, y el punto de vista lógico (que dice qué diagramas UML usar, qué mostrar, etc.) es el viewpoint.

Ejemplo:

- View: “diagrama de clases del sistema de ventas”
- Viewpoint: “uso UML, muestro clases de dominio, omito infraestructura, target: devs”

Views and Beyond

Muchas maneras

- ▶ Hay diversos “estándares” para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

- Es un método práctico para documentar arquitecturas usando el enfoque de views and viewpoints.
- Propone tres categorías de vistas:
 - **Módulo:** cómo se **organiza** el software en unidades de **código**.
 - **Componente y conector:** cómo **interactúan** los elementos en tiempo de ejecución.
 - **Asignación:** cómo se mapean esos elementos al **hardware, equipos o procesos**.
- El “beyond” del título señala que además de vistas, hay que incluir **información adicional:** escenarios, decisiones arquitectónicas, justificaciones, etc.

Es como un “manual práctico” para aplicar la idea de views and viewpoints.

IEEE 1471 (ahora ISO/IEC/IEEE 42010)

Muchas maneras

- ▶ Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

- IEEE 1471: reemplazado por 42010 en 2021
- Fue el **primer estándar formal** de descripción de arquitectura de software (año 2000).
- Introdujo oficialmente los conceptos **de views y viewpoints**.
- Objetivo: que las **arquitecturas sean documentadas de forma consistente y comprensible** para distintos stakeholders.
- Evolucionó al estándar actual ISO/IEC/IEEE 42010, que amplía y formaliza la práctica.
- ISO/IEC/IEEE 42010:2011: primer versión.
- ISO/IEC/IEEE 42010:2022: segunda versión y estándar vigente.

Modelo 4+1 de Kruchten

- ▶ Describir una arquitectura implica describir decisiones en:
 - Código
 - Performance
 - Flujos de datos
 - “Estilo”
 - Funcionalidad
 - ...

Muchas maneras

- ▶ Hay diversos “estándares” para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

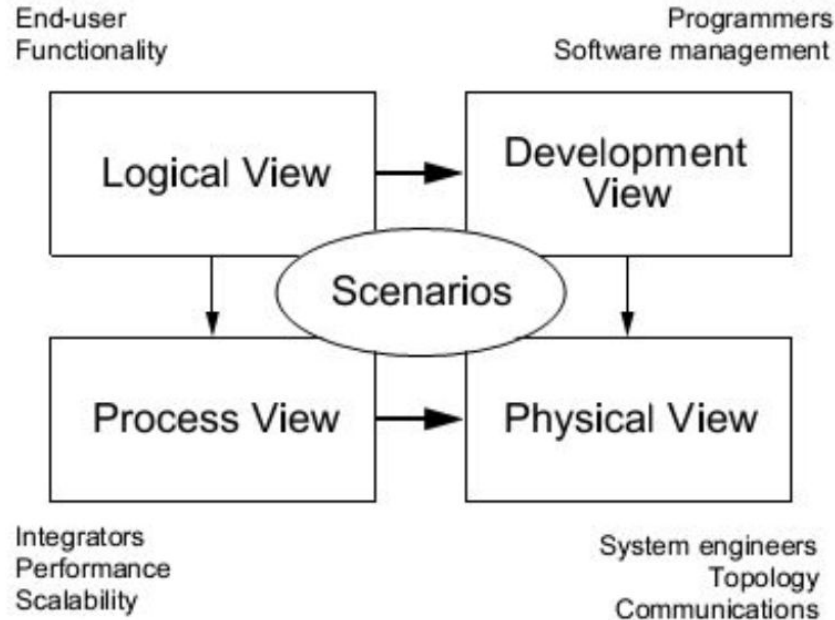
Modelo 4+1 de Kruchten

- ▶ Kruchten propone utilizar 4 vistas principales:
 - Vista lógica
 - Modelo de dominio.
 - DER.
 - Vista de proceso
 - Concurrencia.
 - Sincronización.
 - Vista de desarrollo
 - Organización estática del software en desarrollo.
 - Librerías.
 - Vista física
 - Mapeo entre el software y el hardware.
 - Distribución del hardware.

Muchas maneras

- ▶ Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Modelo 4+1 de Kruchten



Muchas maneras

- Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Modelo 4+1 de Kruchten

- ▶ Cada vista ataca un problema particular.
- ▶ Cada vista refiere a un stakeholder.
- ▶ Cada vista permite un diagrama distinto.
 - Vista lógica.
 - Sequence, communication, class
 - Vista de proceso
 - Activity
 - Vista de desarrollo
 - Package, component
 - Vista física
 - Deployment

Muchas maneras

- ▶ Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Modelo 4+1 de Kruchten

- ▶ Falta la vista “+1” o escenarios
 - Se utilizan casos de uso, por ejemplo.
 - Valida la arquitectura.
 - Explica cómo interactúan los objetos y procesos en un caso típico.

Muchas maneras

- ▶ Hay diversos “estándares” para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Vista Lógica y Vista de Procesos

Muchas maneras

- ▶ Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Vista lógica

- Pregunta: ¿Cómo está **organizado** el sistema desde el punto de vista de las **funcionalidades**?
- Foco: Estructura del software y sus **elementos estáticos**.
- Contenido típico:
 - Clases, objetos, módulos, capas.
 - Relaciones de herencia, composición, asociaciones.
 - Diagramas UML de **clases** o diagramas de **componentes**.

Vista de procesos

- Pregunta: ¿Cómo se **comporta** el sistema en tiempo de **ejecución**?
- Foco: Aspectos **dinámicos**: concurrencia, comunicación, sincronización, rendimiento.
- Contenido típico:
 - Procesos, hilos, tareas concurrentes.
 - Mensajes, colas, comunicación entre procesos.
 - Diagramas UML de **secuencia**, de **actividades**, o de **comunicación**.

Vistas de Desarrollo y Vista Física

Muchas maneras

- ▶ Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Vista de desarrollo (o de implementación)

- Pregunta: ¿Cómo se **organiza** el software en el entorno de **desarrollo**?
- Foco: Estructura estática a nivel de módulos, componentes y paquetes, pensada **para los desarrolladores**.
- Contenido típico:
 - Paquetes, bibliotecas, frameworks.
 - Organización en repositorios o directorios.
 - Dependencias entre módulos.
 - Diagramas UML de **componentes** o de **paquetes**.

Vista física (o de despliegue)

- Pregunta: ¿Dónde y cómo se ejecuta el sistema en la **infraestructura física**?
- Foco: Asignación del software al **hardware** y **redes**.
- Contenido típico:
 - Nodos, servidores, contenedores, dispositivos.
 - Topología de red, balanceadores de carga.
 - Diagramas UML de **despliegue**.

Vista de Escenarios

Muchas maneras

- ▶ Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- ▶ Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- ▶ Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Vista de escenarios (o casos de uso)

- Pregunta: ¿Qué comportamientos clave **validan que las otras vistas funcionan en conjunto?**
- Foco: Historias concretas de uso que atraviesan las demás vistas.
- Contenido típico:
 - Casos de uso detallados.
 - Diagramas UML de **casos de uso** o **secuencia**.
 - Narrativas de interacción usuario–sistema.

Modelo 4+1 de Kruchten

► Consideraciones

- El modelo se adecua a cada caso (tailoring).
 - Suprimir o combinar vistas
 - Utilizar distintos diagramas
- Proceso iterativo.
 - La arquitectura evoluciona y su descripción también.
- Complejidad e interrelación
 - Las vistas son complejas y se relacionan
 - Se deben evitar contradicciones

Muchas maneras

- Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Modelo 4+1 de Kruchten

► Esqueleto propuesto por Kruchten

Title Page
Change History
Table of Contents
List of Figures
1. Scope
2. References
3. Software Architecture
4. Architectural Goals & Constraints
5. Logical Architecture
6. Process Architecture
7. Development Architecture
8. Physical Architecture
9. Scenarios
10. Size and Performance
11. Quality
Appendices
 A. Acronyms and Abbreviations
 B. Definitions
 C. Design Principles

Muchas maneras

- Hay diversos "estándares" para documentar
 - Views and viewpoints
 - Views and beyond
 - Modelo 4+1
 - IEEE 1471
- Son todos parecidos
 - Involucran distintas vistas.
 - Cada vista tiene un foco distinto.
- Veremos 4+1 como ejemplo
 - Tarea para el hogar, leer el paper (Philippe Kruchten).

Documentación con Vista 4+1



¿CÓMO DESCRIBÍS NETFLIX?

Mismo sistema, distintas vistas.

1 VISTA LÓGICA (FUNCIONAL)

Qué hace el sistema y cómo se organiza funcionalmente:



IDEA CLAVE:

qué partes tiene el sistema y para qué sirven

2 VISTA DE PROCESOS (RUNTIME)

Qué pasa cuando apretás "Play":



ASPECTOS:



conurrencia
(millones de usuarios)



comunicación
entre servicios



latencia

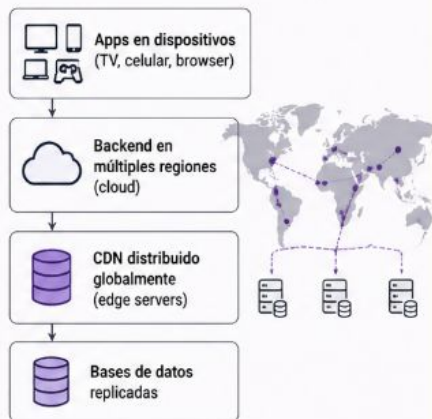


IDEA CLAVE:

cómo se comporta el sistema en ejecución

3 VISTA FÍSICA (DESPLIEGUE)

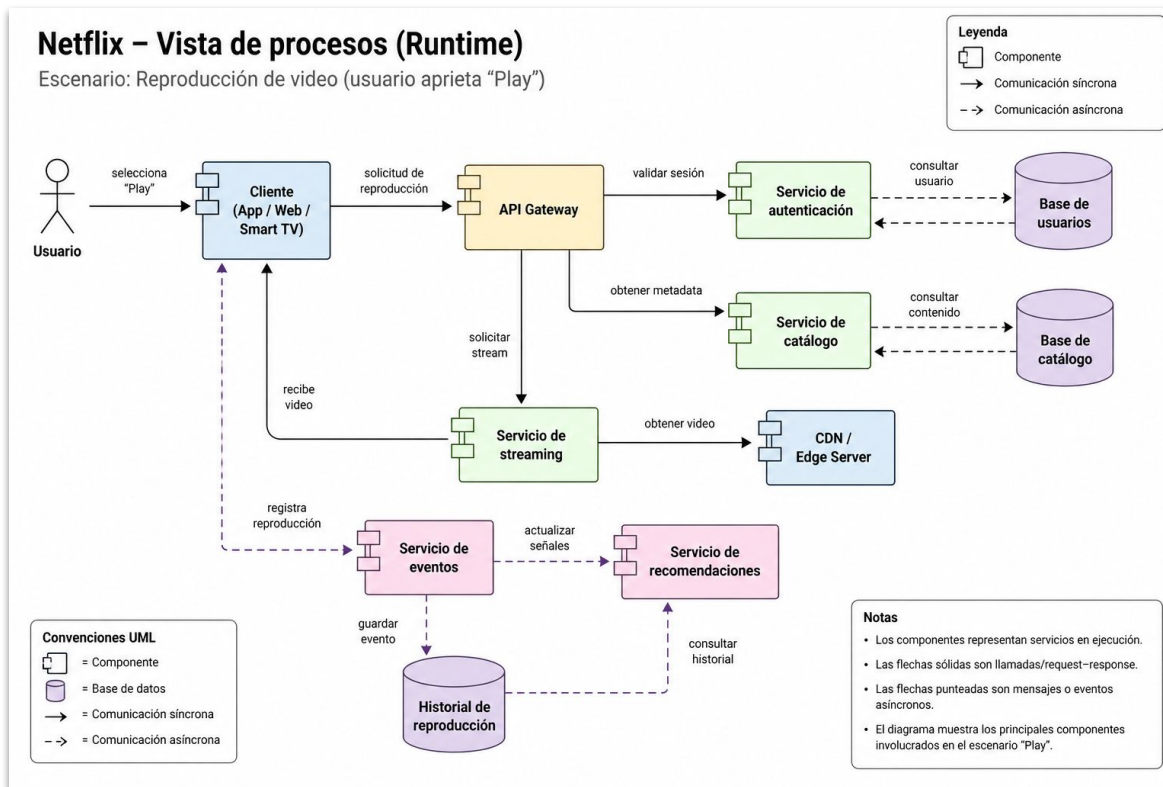
Dónde corre todo eso:



IDEA CLAVE:

en qué infraestructura vive el sistema

Documentación con Vista 4+1 → Uso de UML



Actividad

https://www.researchgate.net/publication/282847758_Multi-View_Software_Architecture_Design_Case_Study_of_a_Mission-Critical_Defense_System

Identificar a qué vista corresponden las 10 figuras

Resumen:

- Lógica = estructura funcional.
- Procesos = ejecución dinámica.
- Desarrollo = organización de código.
- Física = despliegue en hardware.
- Escenarios = validación de todas las anteriores.



	Lógica	Desarrollo	Procesos	Física	Escenarios
Figura 1: Mine Neutralization System main components	X				
Figura 2: High-level use cases					X
Figura 3: Conceptual view (MVC aplicado al sistema)	X				
Figura 4: Star-controller architectural style		X			
Figura 5: Mine Neutralization Vehicle subsystems	¿X?	¿X?			
Figura 6: Layered architecture of the MNS	¿X?		¿X?		
Figura 7: Domain model of the MNS	X				
Figura 8: Detection and classification sonar suite high-level design	X				
Figura 9: MNS controller high-level design	X				
Figura 10: High-level statechart of vehicle control operation			X		

De la documentación al diseño de la Arquitectura

- El Modelo 4+1 → 4 vistas técnicas + escenarios que validan todo
- Describimos la arquitectura
- ¿Cómo la diseñamos? → Estilos

Estilos Arquitectónicos

- ▶ Un estilo arquitectónico es el conjunto de decisiones de diseño aplicables en un contexto particular que restringen las decisiones de diseño en pos de un beneficio particular.
- ▶ Seguir un estilo provee consistencia y previsibilidad.
- ▶ Pueden combinarse múltiples estilos dentro de un mismo sistema, si aplican a contextos distintos.

Dataflow

- ▶ Dominio que define como los datos fluyen por el sistema y son procesados / transformados.
- ▶ Estilos
 - Batch Secuencial
 - Pipes & Filters
 - Hierarchical Layers

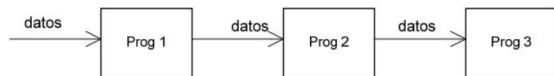
Dataflow en modo *batch secuencial*

Compiladores clásicos (ej. GCC, Clang, JavaC)

- Pipeline secuencial: Análisis léxico (scanner) → Análisis sintáctico (parser) → Análisis semántico → Optimización → Generación de código.
 - Cada fase consume el **output de la anterior**, de manera estrictamente secuencial
-
- Problema: **procesar grandes volúmenes de datos offline**
 - Cómo: etapas una detrás de otra
 - Cuándo usar:
 - ETL
 - procesamiento nocturno
 - pipelines donde no importa la latencia
-
- Trade-off: simple pero lento (alta latencia)

Dataflow – Batch Secuencial

- ▶ Los pasos de procesamiento son programas independientes
- ▶ Cada paso se completa antes de comenzar el que sigue
- ▶ Los datos se transmiten entre programas como un todo, en general mediante archivos.



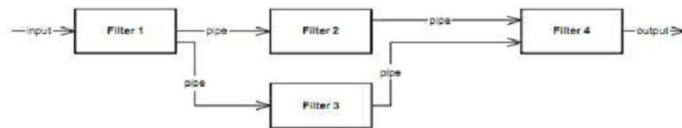
Dataflow en modo *Pipes & Filters*

Renderizado y edición de imágenes / vídeo

- Los editores aplican transformaciones encadenadas: Imagen → Filtro de contraste → Filtro de nitidez → Reducción de ruido → Guardado.
 - En algunos programas, los filtros son “streaming”: producen resultados parciales visibles mientras procesan.
- Problema: **procesar datos en flujo continuo**
 - Cómo: filtros independientes conectados
 - Cuándo usar:
 - streaming
 - sistemas que requieren paralelismo
 - Apache Camel, Spring Integration

Dataflow – Pipes & Filters

- ▶ Hincapie en la transformacion incremental de datos
- ▶ Los filtros transforman sin retener informacion de estado, y no es necesario terminar un paso para empezar el que sigue
- ▶ Los pipes mueven datos entre filtros, pero permiten flexibilizar conexiones.



- Trade-off: más flexible, más complejo de coordinar

Dataflow en modo *Hierarchical Layers*

Protocolos de red (**modelo OSI o TCP/IP**)

- El clásico ejemplo de capas jerárquicas:
Aplicación → Transporte → Red → Enlace → Física.
- Los datos bajan capa por capa, cada una agrega su header o transforma la información, y luego suben en el receptor deshaciendo las transformaciones.

- Problema: **organizar complejidad**

- Cómo: cada capa abstrae a la inferior

- Cuándo usar:

- sistemas grandes

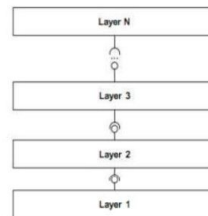
- arquitecturas enterprise

- separación de responsabilidades

- Trade-off: claridad vs posible impacto en performance

Dataflow – Hierarchical Layers

- ▶ Cada capa
 - Provee servicios a la capa superior
 - Es cliente de la capa inferior
- ▶ No todos los sistemas pueden armarse en capas
- ▶ La mayor dificultad es encontrar el nivel adecuado de abstracción



Dataflow – Hierarchical Layers

► Ventajas

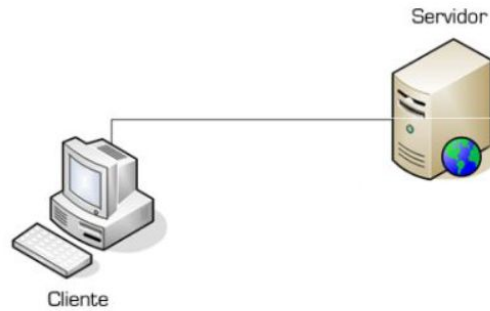
- Permite dividir un sistema complejo mediante
- abstracción.
- Mejoras en una capa impactan en todo el sistema
- Suponen reusabilidad y fácil intercambio

► Desventajas

- No todos los sistemas se pueden estructurar así.
- Posible deterioro de performance con muchas capas.
- Cambios suelen tener que replicarse en todas las capas

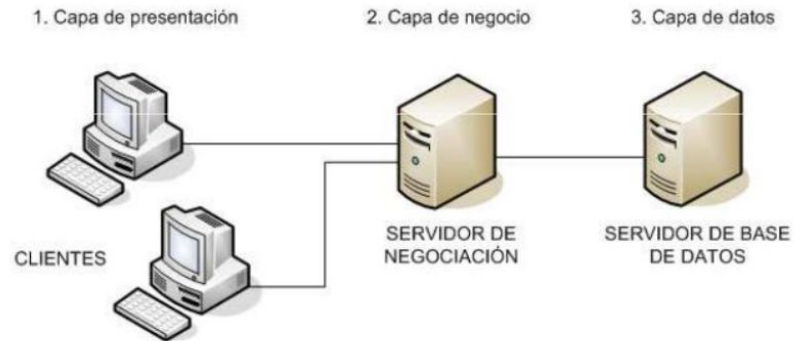
Dataflow – Hierarchical Layers

- ▶ 2 capas – cliente / servidor



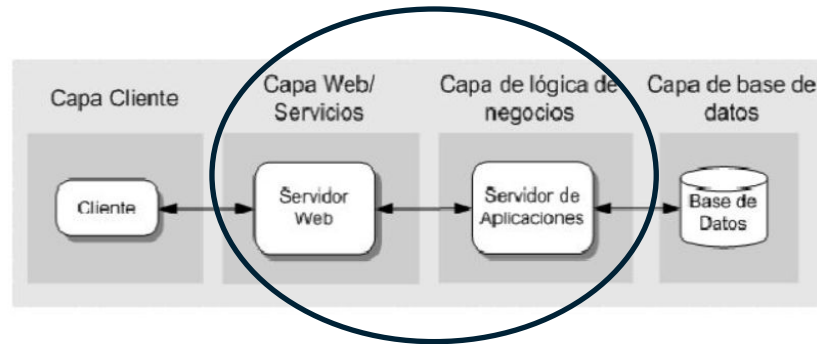
Dataflow – Hierarchical Layers

- ▶ 3 capas



Dataflow – Hierarchical Layers

- 4 capas



Cómo elegir un estilo Dataflow

- ¿Tengo datos en **lotes grandes**? → Batch. Palabra clave: “**volumen**”
 - Sistemas de facturación (SAP, sistemas bancarios)
 - Liquidaciones, cierres diarios, cálculo de intereses: todo corre en batch nocturno.
 - Motores de recomendación offline (ej. Netflix en parte de su pipeline)
- ¿Tengo **flujo continuo**? → Pipes & Filters. Palabra clave: “**flujo**”
 - Apache Kafka:
 - Filters: Los servicios o aplicaciones de procesamiento de streams que leen un mensaje, realizan una operación específica (validar, transformar, enriquecer o filtrar) y escriben el resultado.
 - Pipes: Son los topics. Transportan los flujos de mensajes de un filtro a otro de forma desacoplada y persistente
- ¿Necesito **organizar el sistema**? → Capas. Palabra clave: “**organización**”
 - Hardware → kernel → servicios → aplicaciones.
 - Aplicaciones web típicas (por ejemplo con Django o Spring Framework)
 - Android: Framework → runtime → kernel → hardware

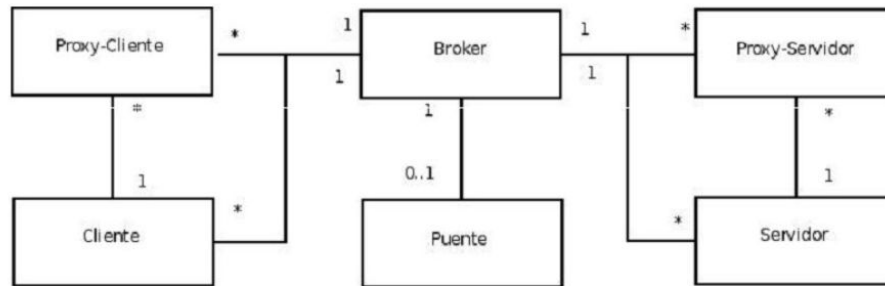
Sistemas Distribuidos

- ▶ Dominio que define cómo se descubren miembros de la red y cómo se comunican.
- ▶ Estilos
 - Broker
 - Publish – Subscribe
 - Forwarder – Receiver
 - Client – Dispatcher – Server

Sistemas Distribuidos – Broker

► Broker

- Un componente “Broker” coordina la comunicación entre clientes y servidores.
- Los clientes y servidores utilizan componentes “proxy” para comunicarse con el broker.
- Permite que la distribución de la aplicación sea transparente para los implementadores.



Broker

- Hay un intermediario central
- Coordina productores y consumidores
- Ejemplo: WhatsApp, email

Problema que resuelve: **desacoplar y enrutar**

Aplicaciones de usuario finales basadas en Broker

Plataformas de **mensajería instantánea**

- WhatsApp, Telegram, Slack, Discord: Los **mensajes no van directo de usuario a usuario**, pasan por brokers (servidores intermedios) que los enrutan, almacenan temporalmente, y entregan según disponibilidad.

Aplicaciones de delivery

- Uber, Rappi, PedidosYa: **El broker aquí conecta pasajeros con conductores, o usuarios con repartidores.** La app del cliente hace la solicitud → broker central recibe → busca un proveedor disponible → reenvía la tarea.

Servicios de **correo electrónico**

- Gmail, Outlook, Yahoo Mail: El broker de mensajería (SMTP/IMAP servers, colas internas) **enruta mails entre dominios y usuarios.**

Sistemas Distribuidos – Publish – Subscribe

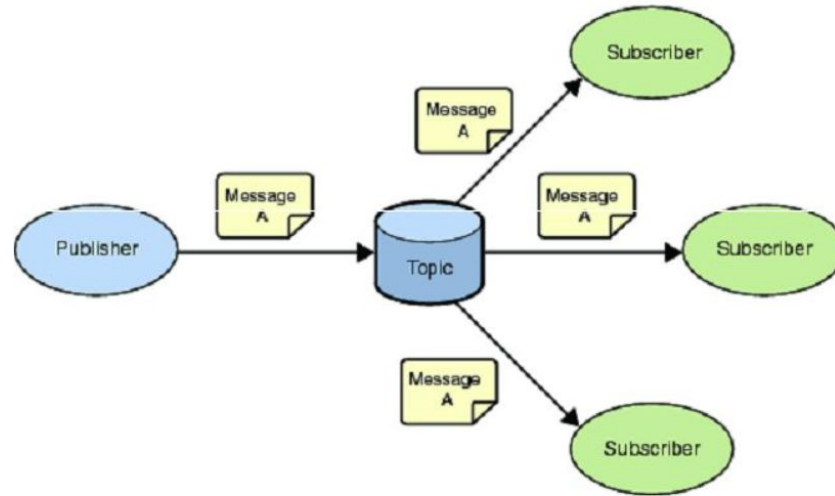
- ▶ Mantiene la sincronización entre componentes cooperativos.
- ▶ Un componente “publisher” notifica a muchos consumidores “subscribers” suscriptos a él.
- ▶ Análogo al patrón de diseño “Observer”.

Publish – Subscribe

- Productores publican eventos
- Consumidores se suscriben
- No se conocen entre sí

Problema que resuelve: **desacoplamiento** total + **escalabilidad**

Sistemas Distribuidos – Publish – Subscribe



Aplicaciones de usuario finales basadas en Publish–Subscribe

Redes sociales

- Twitter/X, Instagram, Facebook: **Seguir a alguien** = **suscribirse** a sus publicaciones.

IoT (Internet of Things)

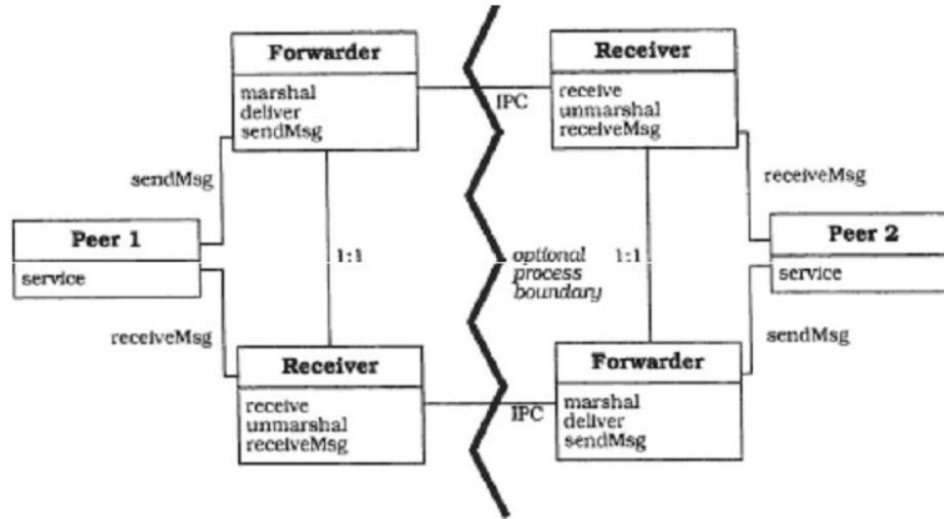
- MQTT es un protocolo pub-sub muy usado.
- Ejemplo: **sensores de temperatura publican datos** en un tópico, y diferentes aplicaciones (monitoreo, control, dashboards) se suscriben.

Logística y transporte

- Sistemas tipo Uber Eats / PedidosYa: Los **repartidores se suscriben a eventos de pedidos** en su zona.
- Cuando un pedido se publica, solo los suscriptores relevantes lo reciben.

Sistemas Distribuidos – Forwarder – Receiver

- Encapsula la comunicación.



Forwarder–Receiver

Peers: Son los componentes de la aplicación que necesitan comunicarse. **No saben cómo viaja la información**, solo invocan al Forwarder para enviar y escuchan al Receiver para recibir.

Forwarder: Se encarga de **recibir** el mensaje del Peer de origen, **transformar** los datos a un formato apto para la red (serialización) y **enviarlos** a través del canal de comunicación.

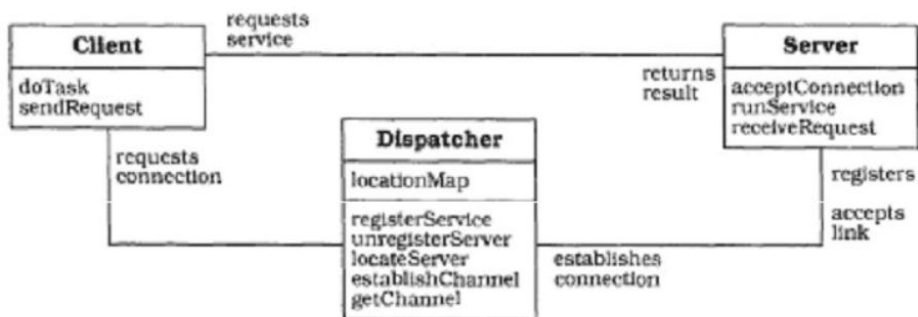
Receiver: Se encarga de **escuchar** el canal de comunicación, **recibir** los bytes binarios, **transformarlos** de vuelta a un formato que el Peer de destino entienda (deserialización) y **entregárselos**.

Problema que resuelve: **transparencia** en sistemas distribuidos para comunicación directa

Sistemas Distribuidos

Client – Dispatcher – Server

- ▶ Provee transparencia en la ubicación.
 - Ejemplo: JNDI, Skype



Client – Dispatcher – Server

- Un dispatcher decide a qué servidor ir
- Balancea carga

Problema que resuelve: **distribución y disponibilidad**

Aplicaciones de usuario finales basadas en Client – Dispatcher – Server

Balanceadores de carga (**Load Balancers**)

- Nginx, HAProxy, AWS ELB, Google Cloud Load Balancer.
- Los clientes hacen la petición al balanceador (dispatcher).
- El **balanceador la distribuye** a uno de varios servidores disponibles.

Sistemas de **bases de datos distribuidas**

- Ejemplo: Oracle RAC, MongoDB Sharded Clusters.
- Un **coordinador/dispatcher recibe queries y decide a qué nodo** de datos enviarlas.

Sistemas Distribuidos

Todos buscan lo mismo: desacoplar + escalar + distribuir

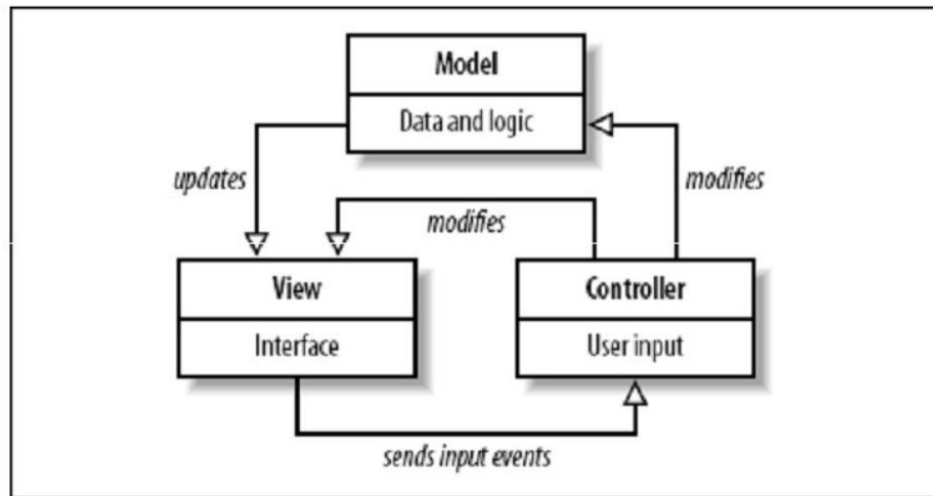
Sistemas Interactivos

- ▶ Mayoría de software tiene algún tipo de interacción con el usuario
 - El objetivo es separar la funcionalidad principal de las vistas de presentación.
 - Estilos principales:
 - MVC (veremos este)
 - PAC
 - MVP
 - VIPER

Sistemas Interactivos – MVC

- ▶ Origen a principios de 80s para Smalltalk
- ▶ Divide la aplicación en 3 tipos de componentes:
 - Modelos
 - Vistas
 - Controladores
- ▶ Modelo: principal funcionalidad e información
- ▶ Vista: muestra información al usuario
- ▶ Controlador: controla los inputs del usuario

Sistemas Interactivos – MVC



Sistemas Interactivos – MVC

▶ Ventajas:

- Múltiples vistas del mismo modelo
- Vistas sincronizadas
- Base potencial para construir un framework

▶ Desventajas:

- Número de actualizaciones potencialmente alto
- Alto acoplamiento entre los tres componentes

Event Based Systems

El sistema es subdividido en múltiples componentes individuales que se comunican entre ellos.

Conceptualmente existen 3 tipos de componentes:

- Event Producers
- Channel
- Event Consumer

El channel puede tomar múltiples formas, conexiones directas (sobre HTTP, TCP/IP, u otro protocolo), a través de un Bus, usando PubSub, etc.

Event Based Systems

Puede tomar múltiples flavors:

- Single Event Processing
- Event Stream Processing
- Complex Event Processing
- Online Event Processing

Estilos de procesamiento de eventos

Tipo de procesamiento	Definición	Ejemplo
Single Event Processing	Procesa eventos <u>de a uno</u> , en forma aislada.	Sensor IoT que dispara alarma si supera temperatura Cajero automático: retiro → valida saldo → entrega dinero
Event Stream Processing	Procesa un flujo <u>continuo</u> de eventos , aplicando transformaciones, agregaciones o filtros en línea.	Google Analytics (realtime): clicks en vivo Twitter: timeline en tiempo real Uber: tracking de viajes en vivo Tecnologías típicas: Apache Kafka
Complex Event Processing	Detecta patrones complejos a partir de <u>múltiples eventos simples</u> , correlacionados en tiempo y espacio.	Bancos: fraude (muchos retiros pequeños + ubicación inusual) → alerta Seguridad: múltiples intentos de login fallidos → bloqueo
Online Event Processing	Procesa eventos <u>en tiempo real</u> , casi sin latencia, para reaccionar inmediatamente al suceso.	Amazon: agregás al carrito → se actualiza al instante Sistemas de trading: compra/venta en milisegundos

¿Dónde aparece el procesamiento de eventos hoy?

Arquitecturas modernas basadas en eventos

- Microservicios
- Sistemas en tiempo real
- Plataformas de streaming (Play → evento, Pause → evento, Skip → evento, Fin → evento...)
- IoT

Ejemplos:

- Netflix: eventos de reproducción, pausas, búsquedas
- Amazon: eventos de compra, pagos, envíos
- Uber: solicitudes de viaje, ubicación en tiempo real
- Spotify: reproducciones, skips, recomendaciones

Event Based Systems

▶ Ventajas:

- Los subsistemas son altamente independientes: se desarrollan, escalan y ponen en producción en forma independiente.

▶ Desventajas:

- Reproducir situaciones particulares / determinar la causa raíz de un problema puede ser complejo.
- Rastrear una operación end-to-end o garantizar tiempo de procesamiento requiere

Otros estilos

- ▶ Existen muchos otros estilos arquitectónicos
- ▶ Algunas categorías:
 - Repositorios y Blackboards
 - Virtual Machines – Interpreters
 - Rule Based Systems
 - Web Server Patterns
 - Concurrency Patterns

Cierre

- Arquitectura de software = vistas + estilos
- Vistas: Cómo describir el sistema
 - Diferentes perspectivas
 - Cada una responde a una pregunta distinta
 - Ejemplo: lógica, procesos, desarrollo, física, escenarios (casos de uso)
- Estilos arquitectónicos: Cómo diseñar el sistema
 - Conjunto de decisiones de alto nivel
 - Restringen y guían el diseño
 - Ejemplo: layers, pipes & filters, broker, pub-sub
- Mismo sistema
 - Distintas preguntas → distintas vistas
 - Distintas decisiones → distintos diseño

Bibliografía recomendada

► Modelo 4+1

- Architectural Blueprints—The “4+1” View Model of Software Architecture

<http://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf>

- Philippe Kruchten
- IEEE Software 12 (1995)

► Patrones de Arquitectura

- Patterns of Enterprise Application architecture.
 - Fowler, Martin
 - Addison-Wesley (2002)